RTSpMSpM: Harnessing Ray Tracing for Efficient Sparse Matrix Computations

Hongrui Zhang
University of California, Riverside
Riverside, USA
hongruiz@ucr.edu

Yunan Zhang
Google
Mountain View, USA
yunandrew@google.com

Hung-Wei Tseng
University of California, Riverside
Riverside, USA
htseng@ucr.edu

Abstract

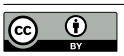
The significance of sparse matrix algebra pushes the development of sparse matrix accelerators. Despite the general reception of using hardware accelerators to address application demands and the convincement of substantial performance gain, integrating heterogeneous hardware accelerators always introduces manufacturing costs on new hardware and challenges the system interconnects.

Inspired by the similar algorithmic behaviors between ray tracing and sparse matrix problems, this paper exploits ray tracing hardware. This increasingly popular accelerator has become the standard in modern GPU architectures to address the demand for virtual/augmented/mixed reality applications. We propose an algorithm that maps the most classical sparse matrix multiplication problem (SpMSpM) as a ray tracing problem. By implementing the proposed algorithm using a modern ray tracing programming framework, the resulting program, `SW-RTSpMSpM` can leverage the accelerated intersection test unit in commercialized GPUs and reveals 1.85× speedup over the state-of-the-art SpMSpM library.

Despite the "similarities" in both problems enabling the potential of accelerating SpMSpM using existing ray tracing hardware, the "difference" between these two algorithms leaves room for performance gain with architectural optimizations. The insights from evaluating `SW-RTSpMSpM` guide the proposal of `RT+SpMSpM`, where we present two major architectural optimizations that (1) allow simultaneous computation on multiplications along with intersection tests and (2) ray mapping and scheduling and a small row accumulation engine to leverage Gustavson's dataflow and eliminate the reliance of additional shader functions and redundant memory operations.

The resulting `RT+SpMSpM` only introduces 0.2% area overhead to a modern high-end ray-tracing-hardware-equipped GPU and improves `SW-RTSpMSpM` by 1.66×, achieving 3.06× speedup over software library and 80% performance per area compared to a state-of-the-art SpMSpM accelerator, without losing the capability in supporting ray tracing.

ACM Reference Format:
Hongrui Zhang, Yunan Zhang, and Hung-Wei Tseng. 2025. RTSpMSpM: Harnessing Ray Tracing for Efficient Sparse Matrix Computations. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3695053.3731072



ISCA '25, Tokyo, Japan

ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731072

1 Introduction

As Moore's Law and Dennard Scaling remain retard, the community moves toward domain-specific accelerators (DSAs) to support the growth of application demands. Beyond the ubiquitous adoption of accelerators for artificial intelligence/machine learning (AI/ML), including Tensor Cores [41, 42] and Tensor Processing Units (TPUs) [21, 22], and ray-tracing hardware for virtual/mixed reality, including RT Cores [41, 42], the trend of developing accelerators for sparse matrix operations emerges due to the significance of high-dimensional, sparse datasets in machine learning, data mining, statistics, and scientific computing. However, besides the overhead of reprogramming, adopting new DSAs into the system also incurs manufacturing costs and idle energy during non-operations. An efficient sparse matrix accelerator can take area overhead equivalent to 26% to 33% of a GPU die [19, 46, 62]. As hardware accelerators communicate with the rest of the host system through system interconnects, integrating new hardware accelerators always increases memory pressure and exchanges data volume, which challenges system interconnecting design.

Instead of designing a separate DSA for sparse matrices, this paper explores an alternative of using and extending ray-tracing hardware that becomes standard integrations in recent graphics processing units' (GPUs) architectures. Ray-tracing hardware and sparse matrix problems can become a good match as ray-tracing algorithms and sparse matrix problems share the following behaviors. First, both ray-tracing algorithms and sparse matrix problems exhibit control flow divergence that challenges conventional GPU architectures. Ray-tracing algorithms trigger different rendering functions depending on the intersection test result between each ray and bounding boxes in the scene. Similarly, sparse matrix computation must test the intersection between non-zero row and column elements and trigger computation only on matched elements. Second, ray-tracing and sparse matrix problems create irregular and indirect memory accesses. In ray-tracing algorithms, irregular memory access patterns emerge when testing the intersection between rays and objects in scenes. Similarly, traversing the compressed data storage formats to match non-zero elements also causes the primary memory bottleneck in sparse matrix applications.

The similarity between sparse matrix computation and ray tracing inspired this paper to propose an algorithm based on the reduction from the most representative sparse matrix multiplication (SpMSpM) problem to a ray-tracing problem. The high-level concept of mapping a sparse matrix multiplication that multiplies an $M \times N$ matrix A by $N \times K$ matrix B and stores the result in an $M \times K$ matrix C to a ray-tracing case is the following. First, we map the non-zero elements in B as objects with a radius smaller than 0.5 in a 2D scene. Second, we go through the non-zero elements in each

row of A and model each as a ray going from one direction. Finally, if there is a hit, a corresponding shader function will perform the required multiplication and accumulation of matching non-zero elements.

By reducing SpMSpM as a ray-tracing problem, SpMSpM can use the hardware-accelerated intersection test in modern ray-tracing hardware to match non-zero elements and trigger the shader function instances. The scheduler associated with modern ray-tracing hardware will optimize grouping shader functions and memory accesses to exploit localities and the single instruction, multiple data (SIMD) parallelism that conventional GPU cores provide. We implemented the proposed SpMSpM algorithm, `SW-RTSpMSpM` and performed experiments on a GPU with NVIDIA's latest generation of RT Cores. The initial software-only, RT-assisted implementation shows that the proposed algorithm can deliver more competitive performance than the highly-optimized `cuSPARSE` library using existing intersection test units and schedulers on RT Cores.

Despite the similarity between SpMSpM and ray tracing, allowing the reduction from SpMSpM to ray tracing, the main difference between SpMSpM and ray tracing creates a performance bottleneck when using ray tracing hardware to assist SpMSpM. In contrast to ray-tracing applications where the shader functions typically contain complex computation that makes sense for conventional GPU cores to perform, SpMSpM's shader function only needs to perform two floating point operations and memory operations. As a result, transferring computation to conventional SIMD cores will lead to inefficiencies. First, both the intersection test units and triggered shader functions on SIMD will generate memory operations for information regarding the coordinates of the matching elements, at least twice the amount of memory accesses compared to pure software SpMSpM, making the already memory-intense SpMSpM problem more memory-bounded. Second, both SIMD and RT cores' shaders must access data and compete for data from the same L1 cache. The uncoordinated small memory requests delay the execution of ray tracing hardware and SIMD cores.

To fundamentally make ray-tracing hardware from a "good" fit to a "perfect" fit of SpMSpM, the ray-tracing hardware should also perform the SpMSpM computation to eliminate the redundant computation and amplification of memory accesses. Fortunately, existing ray-tracing hardware already contains the necessary datapath components to perform the most expensive multiplication that SpMSpM needs, allowing small extensions to the hardware to support the complete SpMSpM computation. Therefore, this paper presents `RT+SpMSpM` that reclaims the unused third-dimension coordinates as a value field of non-zero elements and adds additional logic to allow the element-wise multiplications occur simultaneously with the intersection test without relying on any SIMD core. To efficiently perform accumulations in ray tracing units, `RT+SpMSpM` proposes the use of a ray scheduling policy and the addition of a row cache and an accumulation engine to reduce the complexity of update result values and exploit Gustavson's dataflow for SpMSpM [16].

The software emulation, hardware simulation, and synthesization show that the proposed `RT+SpMSpM` only requires a negligible 0.2% area overhead while preserving the capability of performing ray tracing. `RT+SpMSpM` reveals 1.66× speedup over the software-only, RT Cores-assisted `SW-RTSpMSpM`, and 3.06× over the state-of-the-art `cuSPARSE` library

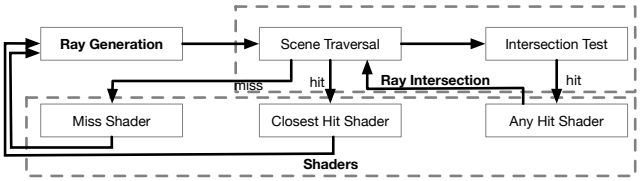


Figure 1: The steps of ray tracing

In summary, this paper makes the following contributions:
- Proposing a novel algorithm to map spare matrix multiplications (SpMSpM) as a ray tracing problem. To the best of our knowledge, this is the first work trying to use ray-tracing hardware for SpMSpM.
- Developing a software-only implementation to accelerate SpMSpM using off-the-shelf ray-tracing hardware, providing an immediate, competitive solution for SpMSpM.
- Observing the performance bottleneck of using ray-tracing hardware to assist SpMSpM.
- Proposing architectural extensions, `RT+SpMSpM`, that enables the complete SpMSpM operations on existing ray-tracing hardware while maintaining the compatibility with ray-tracing applications.

2 Background

`RT+SpMSpM` proposes algorithms and architectural optimizations to make ray-tracing hardware capable of accelerating SpMSpM. This section will briefly overview state-of-the-art ray-tracing hardware and SpMSpM accelerators.

2.1 Ray tracing

Ray tracing is a rendering technique that simulates the path of light in a 3D scene. In contrast to traditional rasterization algorithms that focus on projecting 3D objects onto a 2D screen, ray tracing traces the path of light rays from the eye (or camera) to the scene and back, simulating the physics of light and thus improving visual realism and accuracy.

Figure 1 illustrates the concept of ray tracing. Ray tracing contains three main steps when rendering an image: ray generation, ray intersection, and shading. Some applications may require a ray recursion stage. During the ray generation stage, the algorithm creates a set of rays with their original points and a direction vector that passes through each pixel on the image plane to represent the camera's viewpoint. For each ray, the ray intersection stage will traverse each object from a scene (i.e., scene traversal) and compute if the ray will intersect with objects, which can be spheres, triangles, or polygons, and the intersection point between the ray and the object (i.e., intersection test). Suppose the algorithm identifies an intersection (i.e., ray hit). In that case, the algorithm will trigger shader functions, typically a compute-intensive physically-based rendering or Phong shading function, to simulate the interaction of light with the surface. The algorithm may generate new rays for more complex use cases to simulate indirect lighting (e.g., reflections, refractions) from the intersection point.

Modern ray tracing algorithms and ray tracing hardware accelerators would leverage acceleration structures like Bounding Volume Hierarchies (BVHs) trees to reduce the number of objects

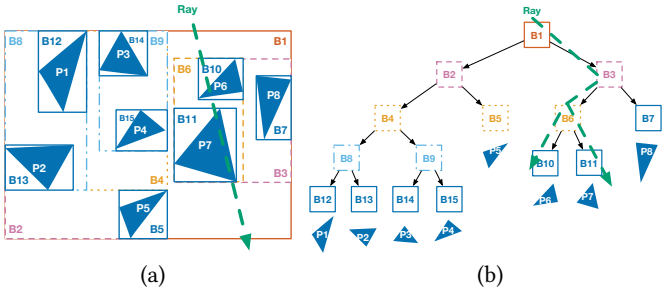


Figure 2: The concept of (a) Bounding Volume Hierarchies (BVH) and (b) the exemplary acceleration structure, BVH tree

to traverse and intersection tests. Figure 2 assumes a scene with eight objects/primitives labeled from P1 to P8. When building the acceleration structure, the building algorithm considers each object as a *Bounding Volume* and gradually groups objects together as a new bounding box at a higher level when inserting new objects into the structure. Once the algorithm finishes building the structure, the ray tracing algorithm or the accelerator can test intersections by traversing the tree. For the BVH structure in Figure 2(a), the building process generates a tree in Figure 2(b). If the ray does not hit the top-level bounding box, B1, the ray intersection test algorithm can declare a miss. Since the exemplary ray in Figure 2(b) hits B1, we will continue the test. As the ray hits B3, the algorithm can ignore all objects that B2 covers. The algorithm will continue until the ray hits the leaf nodes (i.e., B10 and B11) or reaches the last level. Without such an acceleration data structure, a conventional ray tracing algorithm must go through all eight objects and test the intersection conditions.

Despite the high-quality, realistic rendering that ray tracing offers, ray tracing is traditionally inefficient on general-purpose parallel hardware, including GPUs. First, beyond mathematical calculations of complex object geometries, the traversal of BVH trees requires a significant amount of pointer-chasing operations that conventional GPU vector cores cannot efficiently perform [50]. In addition, as the ray tracing algorithm may trigger different compute-intensive shader functions depending on the intersection result, the branch divergence nature makes ray tracing less ideal for conventional GPU cores's microarchitecture.

2.2 Ray tracing hardware

As more applications, primarily virtual/augmented/mixed reality, demand high-quality photorealistic images, modern GPUs incorporate specialized hardware to accelerate ray tracing. Famous examples include NVIDIA's RT Cores [42] and the ray tracing hardware in recent AMD RDNA architectures [3]. Figure 3 depicts the typical integration of ray tracing units in modern GPU architectures [3]. Each unit of hardware that a GPU instruction can control (i.e., Streaming Multiprocessor (SM) in NVIDIA architecture or Compute Unit (CU) in the AMD architecture) will contain three types of computing resources, including single instruction, multiple data (SIMD) cores that are conventional vector ALUs, matrix processors or MXUs (i.e., Tensor Cores in NVIDIA architecture or Matrix Core

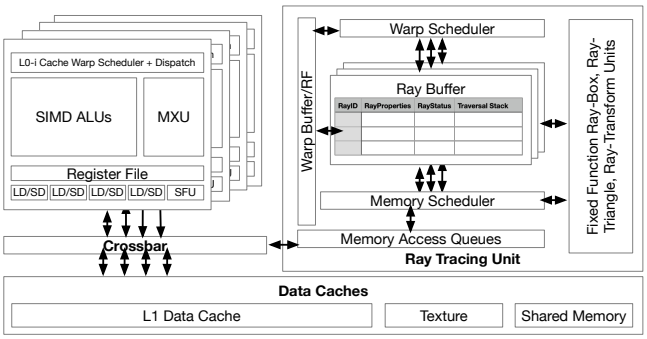


Figure 3: The integration of ray tracing units in modern

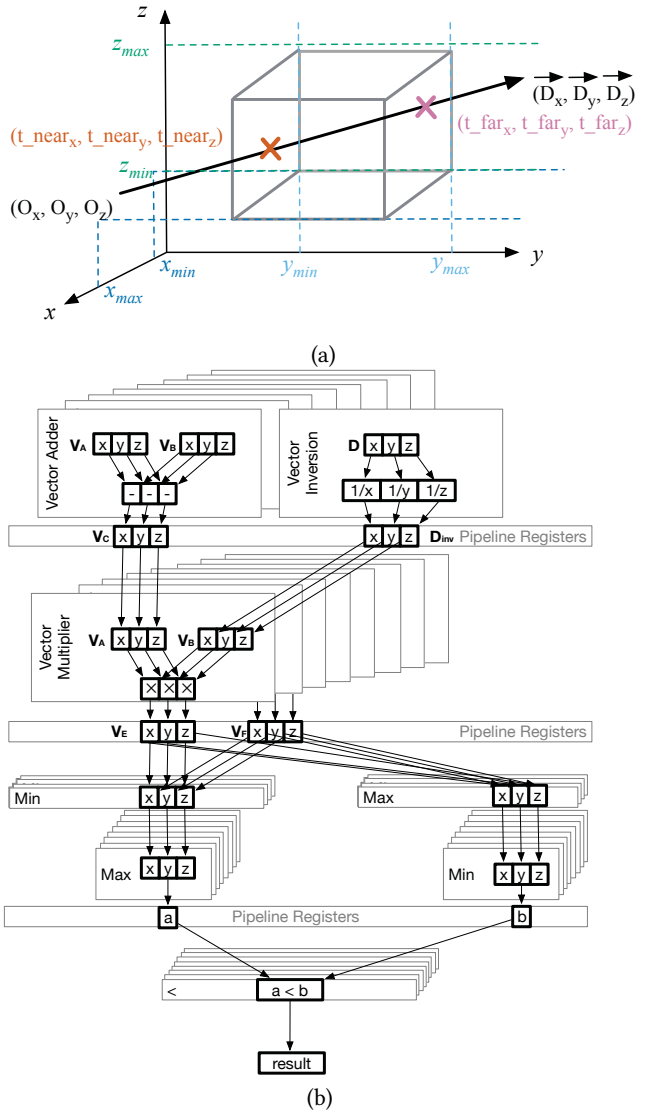


Figure 4: (a) The concept of ray-box intersection test and (b) the excerpted ray-box intersection test pipeline

in AMD architecture), in addition to a ray tracing unit. All computing resources in each basic GPU hardware unit share the same

instruction and memory front ends. Each ray tracing unit contains hardware logic to traverse the BVH tree and a memory scheduler that coalesces and issues access requests of tree nodes to the GPU's memory subsystem. The hardware also contains datapath elements to compute the intersection with each acceleration structure's node without using the conventional GPU cores.

Figure 4(a) illustrates the concept of performing an intersection test between a ray with an origin of (O_x, O_y, O_z) and a bounding box defined by $x = x_{min}$, $x = x_{max}$, $y = y_{min}$, $y = y_{max}$, $z = z_{min}$, and $z = z_{max}$. The goal of the intersection test is to find if the pass-through points (i.e., t_{near} and t_{far}) of the ray and two of the six $x-y$, $y-z$, and $z-x$ planes that bound the box are within the region of the box. Figure 4(b) illustrates the design of a ray-box intersection pipeline using information from API documents of commercialized products [3, 44] and insights from early papers [1, 17, 28, 50]. In the latest RT Cores, the hardware logic also supports ray-triangle intersection tests on 3D coordinates. For ray-box intersection tests, the pipeline includes 3-element vector floating point subtractors, vector multipliers, min/max units, and comparators to perform ray-box intersection tests. For ray-triangle intersection pipeline, the hardware contains multiple 3-element, vector floating point cross and dot product units, subtractors, multipliers, reciprocals, and comparators. According to public die-shot photos of the Ada Lovelace architecture, the RT Core accounts for 18.9% of the area within an SM [30].

2.3 Alternatives

Prior work addresses the demand of SpMSpM through innovative algorithms on other existing parallel hardware than ray tracing units or developing distinct accelerator architectures. However, each has advantages and disadvantages.

Software Sparse Libraries: Libraries like Intel MKL [59], cuSPARSE [39], SPOOLES [4], [10], cuBLAS [18], provide optimized routines for sparse matrix operations on general-purpose hardware. However, these libraries face challenges as they rely on general-purpose hardware, which can lead to inefficiencies in exploiting sparsity patterns for fine-grained optimizations. Furthermore, sparse matrices with irregular sparsity patterns often incur high control divergence, which the SIMD processing model cannot handle efficiently. Unlike prior work exploiting the limitation of SIMD hardware, the proposed software solution repurposes using the acceleration structure traversal and intersection tests in ray tracing hardware. It accelerates sparse computations by mapping them to highly optimized traversal and intersection operations, originally designed for ray tracing, an algorithm with highly control flow divergent, irregular memory locality like sparse matrix operations. This novel use of existing hardware combines the flexibility of software libraries with the performance advantages of hardware acceleration.

Sparse Matrix/Tensor Accelerators: Sparse accelerators are designed to handle irregular data access patterns and computations inherent in sparse workloads, including Cambricon-X [61], felxagon [35], DMSA [36], tardigrade [5]. Prior sparse accelerators also focused on intra-operator reuse for SpMSpM, including DRT [45], OuterSPACE [46], SpArch [64], MatRaptor [52], ExTensor [19], Gamma [62], HSS [60], RM-STC [20], and Sparsepipe [63]. These accelerators improve sparse tensor algebra performance through specialized hardware pipelines and dataflow for efficient computation on sparse data [14, 63]. While sparse accelerators deliver high performance for specific workloads, there is a significant increase in hardware costs. In contrast, `RT+SpMSpM` largely utilizes existing ray tracing hardware to accelerate sparse computations without requiring dedicated sparse-specific hardware. This approach avoids costly hardware investments and extends the utility of widely available ray tracing GPUs for sparse matrix operations.

```
Function SWRTSpMSpM SpM Mat_A, SpM Mat_B, BVHTree tree
    tree = buildAccelerationStructure(Mat_B);
    rows_Mat_B = Mat_B's number of rows;
    forall non-zero elements p in Mat_A do
        r = createRayFromAnElement(p);
        intersectionTest(r,tree,MAC,NULL);
    end
end
```

Algorithm 1: Ray tracing based SpMSpM Overview

3 Mapping SpMSpM to a Ray-Tracing Problem

The fundamental motivation and the core innovation behind exploring ray tracing hardware to accelerate SpMSpM is the feasibility of transforming SpMSpM into a typical ray tracing problem. This section will provide an example to illustrate the concept and an overview of the proposed algorithm.

3.1 The high-level concept

The algorithmic process of performing ray tracing and SpMSpM is similar. In ray tracing, the algorithm finds the intersection between each ray and objects and triggers corresponding shader functions. In SpMSpM, the algorithm finds the intersection between non-zeros with index matches in one dimension and triggers multiplications and accumulations accordingly.

Figure 5 uses an example to illustrate the concept of formulating SpMSpM as a ray tracing problem. Figure 5 assumes two input matrices, an 8×4 matrix A and a 4×8 matrix B. ① We map the three non-zero elements in B from their coordinates in the matrix to the corresponding location in 2D $x-y$ plane as the scene objects at $(0,1)$, $(1,4)$, and $(2,7)$. ② For each non-zero element in A, $(2,1)$, $(2,3)$, $(3,3)$, and $(7,1)$, we generate 4 rays using their row indexes as $x = 1$, $x = 1$, $x = 3$, and $x = 4$, correspondingly. ③ Then, we can run a ray tracing algorithm after getting the list of rays and objects. When performing the intersection test for ray $x = 1$ generated by $(2,1)$, we will identify a hit with an object at $(1,4)$. ④ Therefore, the transformed ray tracing algorithm should trigger a shader function that performs multiplication on A's $(2,1)$ and B's $(1,4)$, and finally, ⑤ accumulates the result in C's $(2,4)$. Similarly, after figuring out the ray hit of $x = 1$ from A's $(7,1)$ and B's $(1,4)$, we multiply A's $(7,1)$ and B's $(1,4)$, and accumulate the result to a result matrix's $(7,4)$. For rays $x = 3$ and $x = 4$, the ray tracing will not trigger any computation since there is no object on the ray.

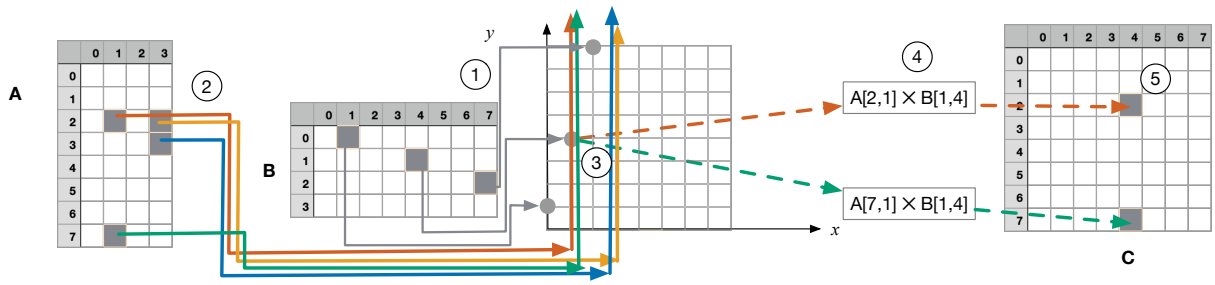


Figure 5: The mapping between SpMSpM and ray tracing

Function ***buildAccelerationStructure*** *SpM Mat*
  AccelerationStructure *tree* = new AccelerationStructure();
  forall *non-zero elements p in Mat* **do**
    x = p's row index;
    y = p's column index;
    createBoundingBox(center = x, y, 0; width = 0.4);
    insertIntoBVHTree(*tree*,*p*);
  end
  return *tree*;
end

Algorithm 2: The function builds acceleration structure from a sparse matrix

Function ***createRayFromAnElement*** *element p, columns_B*
  x = p's row index;
  y = p's column index;
  $ray.id = x$;
  $ray.origin = (y, 0, 0)$;
  $ray.direction$ = (0,columns_B, 0);
  return *ray*;
end

Algorithm 3: The function creates a ray from a non-zero element of a sparse matrix

3.2 The ray tracing algorithm for SpMSpM

Leveraging the similarities, Algorithm 1 lists the pseudo-code of the proposed algorithm. The inputs are two sparse matrices, Mat_A and Mat_B, where Mat_A is an $M \times N$ matrix and Mat_B is an $N \times K$ matrix. The algorithm's output is an $M \times K$ matrix Mat_C. The `SWRTSpMSpM` function controls the execution flow of the main algorithm. The `SWRTSpMSpM` first invokes the `buildAccelerationStructure` function on matrix Mat_B to get a bounding box tree structure *tree*. Then, the main algorithm extracts each non-zero element p from matrix Mat_A and creates a ray r using information from p using the `createRayFromAnElement` function. The algorithm will then perform intersection tests of r and each node in *tree* using the `intersectionTest`. If there is any hit between r and a node in *tree*, the algorithm will trigger a `MAC` function. The algorithm triggers no computation if the ray does not hit any node. Though the Algorithm 1 seems to create rays, perform intersection tests, and trigger hit functions iteration-by-iteration, the actual execution model on the GPU will perform many iterations in parallel.

Algorithm 2 presents the concept of building the acceleration tree structure from a sparse matrix. The `buildAccelerationStructure` function traverses each non-zero element in the input matrix *Mat*. For each non-zero element p at xth row and yth column of the input matrix, the algorithm creates a bounding box at $(x, y, 0)$ coordinate in the bounding box tree structure. Since the coordinates of matrix elements are all integers, the width of each box must be smaller than 0.5 so as not to overlap with another node. In Algorithm 2, we set the box width as 0.4.

After the algorithm builds the acceleration structure from one matrix, the main algorithm extracts each non-zero element from the other matrix. It creates a ray using the `createRayFromAnElement` function in Algorithm 3. The `createRayFromAnElement` function takes a non-zero element p from Mat_A and the number of columns in Mat_B as inputs. The `createRayFromAnElement` extracts p's x-coordinate, x and the y-coordinate, y, to create a ray with an origin from $(y, 0, 0)$, a vertical direction as $(0, columns\_B, 0)$, and an ID of x.

In modern GPU hardware, the ray tracing hardware will perform the `intersectionTest` function once the application provides the rays and the acceleration structure. Therefore, we do not dive into the details of the algorithm that the hardware implements.

4 The performance of the pure software `RT+SpMSpM`

As we reduce SpMSpM as a ray tracing problem, the proposed algorithm in Section 3 can fit modern ray tracing pipelines. Therefore, we present a pure software implementation of the `RT+SpMSpM` algorithm to evaluate the performance of the proposed algorithm on existing ray tracing hardware on modern GPUs. Our implementation of existing ray tracing hardware found that pure software implementation already brings 1.85× speedup compared to a state-of-the-art GPU SpMSpM implementation. However, after investigating the performance of the software implementation, we found that the modern ray tracing pipeline for SpMSpM still has room to improve due to the distinguishing characteristics between ray tracing and SpMSpM. This section will describe the presented software implementation and our experiments in detail.

4.1 `SW-RTSpMSpM` application programming

The software solution presents itself through an application programming interface (API) that hides the implementation details from programmers. The proposed API maintains an interface identical to the existing sparse matrix multiplication APIs that NVIDIA's

```
1: rtSpMSpMHandle_t     handle = NULL;
2: cusparseSpMatDescr_t matA, matB, matC;
   // Create an RTSpMSpM handle
3: CHECK_RTSpMSpM( rtSpMSpMCreate(&handle) )
   // Create sparse matrix A in CSR format
4: CHECK_CUSPARSE( cusparseCreateCsr(&matA, A_num_rows, A_num_cols, A_nnz,
                                     dA_csrOffsets, dA_columns, dA_values,
                                     CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                     CUSPARSE_INDEX_BASE_ZERO, CUDA_R_32F) )
   // Create sparse matrix B in CSR format
5: CHECK_CUSPARSE( cusparseCreateCsr(&matB, B_num_rows, B_num_cols, B_nnz,
                                     dB_csrOffsets, dB_columns, dB_values,
                                     CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                     CUSPARSE_INDEX_BASE_ZERO, CUDA_R_32F) )
   // Create sparse matrix C
6: CHECK_CUSPARSE( cusparseCreateCsr(&matC, A_num_rows, B_num_cols, 0,
                                     nullptr, nullptr, nullptr,
                                     CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                     CUSPARSE_INDEX_BASE_ZERO, CUDA_R_32F) )
   // execute RTSpMSpM
7: CHECK_RTSpMSpM( rtSpMSpM(handle,
                            CUSPARSE_OPERATION_NON_TRANSPOSE,
                            CUSPARSE_OPERATION_NON_TRANSPOSE,
                            &alpha, matA, matB, &beta, matC, CUDA_R_32F,
                            SW, nullptr) )
   // destroy matrix/vector descriptors
8: CHECK_CUDA( cudaMemcpy(matC, dC, C_size * sizeof(float),
                          cudaMemcpyDeviceToHost) )
9: CHECK_CUSPARSE( cusparseDestroySpMat(matA) )
10: CHECK_CUSPARSE( cusparseDestroySpMat(matB) )
11: CHECK_CUSPARSE( cusparseDestroySpMat(matC) )
12: CHECK_CUSPARSE( cusparseDestroy(handle) )
```

Figure 6: The integration of ray tracing units in modern architectures

cuSPARSE and AMD's ROCm supports [43]. Both APIs mentioned above and the proposed API accept the following arguments: (1) a handle that provides runtime context-related and underlying hardware information that upcoming function calls can continuously reuse, (2) information regarding if each matrix is transposed or not, (3) GPU device memory pointers to compressed sparse row (CSR) or compressed sparse column (CSC) formatted input matrices and the output matrix, (4) scaling factors (e.g., α and β) in general matrix multiplications, (5) the data type, (6) the algorithm to perform (e.g., software only for this section and hardware accelerated for the upcoming sections), and (6) a device buffer that the programmer can optionally pass.

Figure 6 illustrates applying the proposed interface in real applications. The code is almost identical to an existing program that calls cuSPARSE functions except for lines 1, 3, and 7. In line 1, we allocate a handler for upcoming RT+SpMSpM function calls, replacing the `cusparseHandle_t` declaration. In line 3, we initialized the handler using the `rtSpMSpMCreate` instead of a `cusparseCreate` function call to detect the underlying hardware and help decide the optimizations that RT+SpMSpM computation may use. Finally, in line 7, we replaced a `cusparseSpMM` function call with our `rtSpMSpM` function call. However, lines 2, 4–6, 8–12 reuse the same code from the original code calling `cuSparse` functions. As each line we modified changes the function call and data types, the code example also reveals that we can quickly leverage RT+SpMSpM by processing the source code using a script without any effort from the programmer.

4.2 Implementation details

We implemented the core of the proposed SpMSpM algorithm as a `rtSpMSpM` library function that accepts two CSR/CSC formatted matrices as inputs and returns the pointer of the resulting matrix. As our testbed uses NVIDIA GPUs, our implementation (i.e., SW-RTSpMSpM) uses API functions from the `optix` library [44] to invoke ray tracing operation that may work on RT Cores. It uses customized CUDA kernels that may invoke CUDA functions for shader functions.

Once receiving the second sparse matrix, SW-RTSpMSpM parses the matrix and invokes `optix`'s built-in `optixAccelBuild` function to create a box object and insert the object into an RT-Cores-friendly acceleration structure. In SW-RTSpMSpM, we implemented CUDA kernels to parse the sparse matrix and insert the value of the sparse matrix into a hash structure. We extract the indexes of non-zero elements from the parser as inputs to the `optixAccelBuild` function, and the function will automatically leverage the hardware acceleration on the GPU card to build the RT-Cores-friendly acceleration structure.

After building an acceleration structure from a sparse matrix, the program will create a pipeline that performs ray generation functions, traversal, intersection tests, and shader functions in parallel. The implementation of the ray generation function is a CUDA kernel function where the function extracts the other sparse matrix and generates a ray through implementing Algorithm 3. SW-RTSpMSpM will bind the any-hit shader as a CUDA kernel that extracts the box data and the ray data from a hit intersection result. The CUDA kernel will fetch the corresponding matrix values to multiply the values and accumulate the result to an entry in a hash structure that stores the result. SW-RTSpMSpM does not use closest-hit function since SW-RTSpMSpM needs to continue traversing the structure along the path of the complete column until the ray hit the last leaf. SW-RTSpMSpM does not use any miss shader function since the algorithm should not perform any computation if there is no hit.

The latest generation of ray tracing hardware can perform the traversal and ray intersection test but still rely on the GPU's SIMD cores (CUDA Cores) to perform the ray generation and shader functions. In other words, the ray tracing hardware will accelerate the `intersectionTest` part in Algorithm 1.

4.3 Experimental setup

We executed the program on a Ubuntu Linux machine with kernel version 6.8.0-48, an Intel Core i7 14700K processor, 128 GB of main memory, and an NVIDIA RTX 4090. The RTX 4090 GPU contains 128 RT Cores, each of which can perform an individual ray intersection test. The CUDA version is 12.5. We select publicly available sparse matrices from real-world datasets [9] that Table 1 lists as our inputs. The selected matrices and the evaluation methodology are the same as the ones that GAMMA [62] use to help us assess the performance of our implementations against the state-of-the-art accelerator. As these matrices represent graphs, they are all square matrices. The densest matrix in this set still has lower than 0.2% of non-zero elements. For each dataset, we multiply the matrix itself. For each experiment, we compare the computing time on CSR formatted matrices using SpMSpM functions. We compare line 7 in Figure 6 for both baseline and SW-RTSpMSpM. As we create BVH trees inside the `rtSpMSpM` function, the measurement also considered the additional overhead the proposed algorithm would introduce.

4.4 Experimental result of the software-only implementation

Figure 7 compares the performance of the software implementation of ray-tracing-hardware-assisted SpMSpM (SW-RTSpMSpM)

Table 1: Characteristics of our datasets (all square)

Datasets from GAMMA [62]									Large Datasets [9]		
Matrix	Rows	Density	Matrix	Rows	Density	Matrix	Rows	Density	Matrix	Rows	Density
roadNet-CA	1,971,281	0.00014%	p2p-Gnutella31	62,586	0.00377%	filter3D	106,437	0.02389%	kmer_V1r	214,005,017	0.00000%
webbase-1M	1,000,005	0.00031%	offshore	259,789	0.00629%	email-Enron	36,692	0.02731%	kmer_U1a	67,716,231	0.00000%
web-Google	916,428	0.00061%	cage12	130,228	0.01199%	ca-CondMat	23,133	0.03493%	stokes	11,449,533	0.00027%
mario002	389,874	0.00138%	2cubes_sphere	101,492	0.01599%	wiki-Vote	8,297	0.15066%	great-britain_osm	7,733,822	0.00003%
amazon0312	400,727	0.00199%	cop20k_A	121,192	0.01786%	poisson3Da	13,514	0.19313%	rajat31	4,690,020	0.00009%
scircuit	170,998	0.00328%									

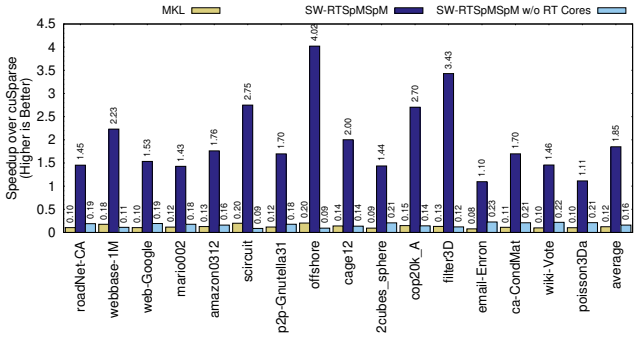


Figure 7: The speedup of SW-RTSpMSpM (w/ RT Cores and w/o RT Cores) over cuSPARSE (the baseline) and MKL

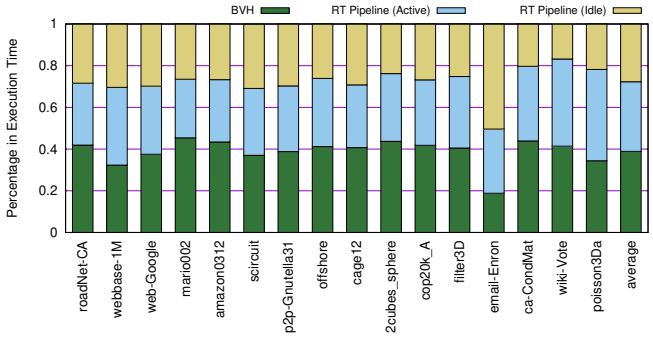


Figure 8: The breakdown of execution time in SW-RTSpMSpM

and the current version of the cuSPARSE library. SW-RTSpMSpM achieves 1.85× speedup over cuSPARSE. We also included intel MKL as another baseline, where the performance is only 12.3% of cuSPARSE and SW-RTSpMSpM outperforms MKL by 14.95× on average. In this paper, we used geometric mean as the averaging method for speedups to discount the outliers.

The performance is synthetic to several factors in the input matrices, including the dimensions, the density, and the depth of the formed BVH tree. In general, SW-RTSpMSpM outperforms cuSPARSE the most when the matrix has reasonably large dimensions but leads to a relatively shallow BVH tree. For matrices where we achieve higher than the average speedup, they all have more than 100,000 rows. However, we also found that as the resulting BVH tree depth from the source matrices determines the performance of the hardware-accelerated ray-tracing algorithms, the ratio between the maximum number of non-zero elements in a row (i.e., *the worst-case row density*) and the dimension of a row affects the performance. For the least performing case, email-Enron, where the matrix has at most 1,245 elements with a row size of 36,692, SW-RTSpMSpM only achieves 1.10× speedup over cuSPARSE. In contrast, filter3D has a similar density as email-Enron, 0.02389% v.s. 0.02731%. However, SW-RTSpMSpM achieves 3.43× speedup over cuSPARSE as the worst case scenario in filter3D has only up to 100 elements within a large row size of 106,437. Another evidence where the depth affects the performance is the comparison between 2cubes_sphere and cage12. Both matrices have similar sizes and densities, but SW-RTSpMSpM achieves higher speedup for cage12 than 2cubes_sphere as 2cubes_sphere may result in 1.75× deeper tree than the other.

Figure 7 also presents the configuration with no hardware acceleration for ray-tracing-related algorithms as "SW-RTSpMSpM w/o RT Cores". This configuration demonstrates the importance of modern ray-tracing accelerators for our proposed algorithm. Without ray-tracing acceleration, the performance is only 16% of the cuSPARSE baseline due to the overhead of BVH tree construction and control divergence of the algorithm.

4.5 Lessons learnt from the software-only implementation

Beyond the success of reducing SpMSpMs as ray tracing problems and achieving 1.85× speedup over a state-of-the-art library, detailed analysis suggests the potential of performance gain if RT Cores can address the deficiencies of using ray tracing hardware for SpMSpM. Figure 8 depicts the execution time breakdown in SW-RTSpMSpM. Despite applications on average spending 39% of execution in building the acceleration structure that many ongoing research projects of ray tracing hardware are improving, this paper will focus on the remaining 61% of time that applications spend on the real ray tracing pipeline. In Figure 8, we found that the pipeline idles 45.3% during the pipeline execution (i.e., 27.7% of the total execution time) of the time due to the memory access latencies. Detailed profiling and simulation of memory accesses reveal the following insights.

First, the modern ray tracing pipeline that only performs intersection tests in RT Cores introduces redundant operations in the shader functions and the amplification of total memory accesses. As these shader functions execute on hardware units separate from RT Cores, these functions must contain 10 ALU operations, of which only two contribute to the outcome, but the rest generate the addresses and indexes. Each shader function that performs an element-wise multiplication and accumulation also triggers four memory accesses to retrieve the hit ray and object information, in addition to the four memory accesses essential for SpMSpM. As SpMSpM is already memory-bounded, the amplification of memory accesses makes the implementation more memory-bandwidth dependent.

The second issue is the contention of memory access. As RT Cores and SIMD cores share the same L1 cache, the intersection tests, and the shader function need to access L1 simultaneously.

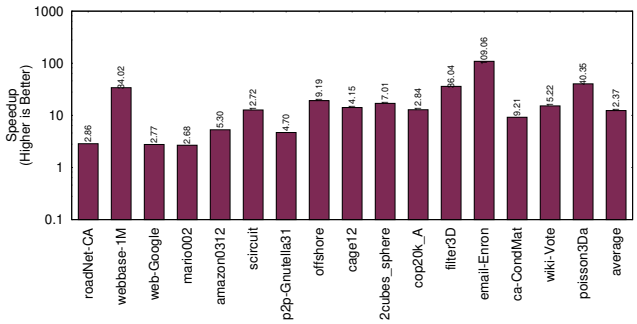


Figure 9: The speedup of an "ideal" ray tracing pipeline execution

Since the shader function is more memory-dependent, the resource contention between uncoordinated memory accesses from RT Cores and SIMD cores further delays the execution of these shader functions.

Figure 9 depicts the "theoretical" speedup of the ray tracing pipeline execution time on the same datasets by emulating the execution of only performing intersection tests. We composed shader functions that only perform increments to a local variable to minimize the memory traffic and arithmetic operations. Without intervention from additional shader functions, the intersection-only pipeline is 12.37× faster than the measured result. Using Amdahl's Law, we can achieve 2.22× speedup over the current software-only version if an "ideal" hardware accelerator presents. Again, the ideal speedup is a synthetic effect from multiple factors, including the sizes, the density, and the worst-case row density, as Section 4.4 explains.

Both issues do not cause performance bottlenecks for conventional ray-tracing applications, as the shader function of most ray-tracing applications is compute-intensive. As a result, the shader function is not memory-bounded, and the amount of memory operations is only a small fraction compared to the application.

5 RT+SpMSpM

RT+SpMSpM extends the existing ray tracing unit with the ability to perform both ray tracing and SpMSpM efficiently. As ray tracing hardware does not initially target SpMSpM, the current SW-RTSpMSpM implementation has to rely on shader functions using conventional SIMD cores to perform multiplications and accumulations. The separation of intersection tests and numerical operations causes inefficiency in SW-RTSpMSpM. The key to eliminating the reliance on external shader functions is to enable multiplications and accumulations in the ray tracing unit.

5.1 Reusing existing multipliers to support multiplications

RT+SpMSpM proposes using existing multipliers in the ray-box intersection test unit, adding wires and multiplexers, and extending the result buffer to fulfill the demand of performing multiplications. As ray-tracing applications require ray-tracing hardware to support intersection tests in three-dimensional space, the vector multiplier of the intersection unit must support simultaneous multiplications

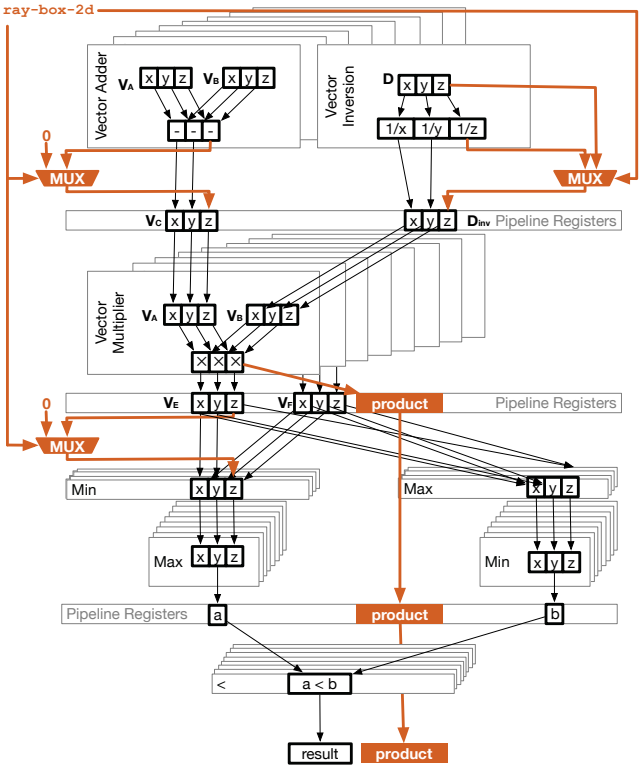


Figure 10: The proposed architectural modifications in the ray-box intersection test pipeline

on two three-element vectors. However, as sparse matrices are only two-dimensional by nature, the proposed SW-RTSpMSpM algorithm always fills zeros in the third dimension but leaves the multiplier for the third dimension performing operations on zeros.

RT+SpMSpM reclaims the wasted multiplier that the third dimension originally used to perform the multiplication. Figure 10 shows the extended ray-box intersection test pipeline in supporting the desired feature. We select the ray-box intersection as ray-box intersection test only needs 4 pipeline stages and has shorter latency. In contrast, ray-triangle test would need 9 pipeline stages. RT+SpMSpM offers a new operation, *ray-box-2d* in the ray-box intersection test pipeline that considers the z-dimension coordinate of the direction of the testing ray and the bounding box as separate values from the coordinate. The signal indicating whether *ray-box-2d* is enabled will feed into three new multiplexers: (1) the first multiplexer is between the ray input to the z-dimensional input of the vector ALU, (2) the second multiplexer is between the vector ALU for the z-dimension output and the following intersection test logic, and (3) the third multiplexer is at the end of testing result output. None of these multiplexers are within the longest vector multiplication stage. Therefore, the proposed design will not increase the cycle time of ray tracing units.

When the pipeline operates in the *ray-box-2d* mode, the first multiplexer will ignore the inverse of the testing ray direction and directly use the z-coordinate from the ray direction as in the

Function ***buildAccelerationStructureRT+SpMSpM*** *SpM Mat*
  AccelerationStructure $tree$ = new AccelerationStructure();
  forall *non-zero elements p in Mat* **do**
    x = p's row index;
    y = p's column index;
    v = p's value;
    createBoundingBox(center = $x - 0.2$, $y - 0.2$, v; width = 0.4);
    insertIntoBVHTree($tree$,p);
  end
  return $tree$;
end

Algorithm 4: The function builds acceleration structure from a sparse matrix

Function ***createRayFromAnElementRT+SpMSpM*** *element p, columns_B*
  x = p's row index;
  y = p's column index;
  v = p's value;
  $ray.id$ = x;
  $ray.origin$ = $(y, 0, 0)$;
  $ray.direction$ = (0,columns_B,v);
  return ray;
end

Algorithm 5: The function creates a ray from a non-zero element of a sparse matrix

multiplier's input. In the meantime, the second multiplexer uses a zero output for the z-dimension in the intersection test and redirects the output as an input to the third multiplexer. If *ray-box-2d* mode is enabled, the third multiplexer will propagate the multiplication result to the result buffer.

The software implementation needs the following modifications in supporting *ray-box-2d* mode. First, the software must turn on the *ray-box-2d* mode before executing the pipeline using a new instruction that `RT+SpMSpM` provides and turn off after SpMSpM to support normal ray tracing. Second, the building primitive of the acceleration structure must ignore the z-coordinate to ensure the most efficient traversal. Third, one side of the bounding box coordinates must align with the exact y-coordinate of the sparse element to ensure the result buffer catches the correct y-coordinate. In other words, instead of making a bounding box centered at the element's coordinates, we need to offset the center by half of the box width. The `buildAccelerationStructureRT+SpMSpM` function in Algorithm 4 summarizes the changes in creating acceleration structures. Finally, the ray generation function must extract elements row-by-row and fill the matrix value as the z-coordinate of the ray direction as Algorithm 5 shows.

5.2 Accumulation engine in ray-tracing unit

In contrast to multiplications that we can simultaneously execute in the intersection test, we have to separate the case of accumulation for the following reasons despite the free adders in the vector ALUs. First, performing accumulation simultaneously with multiplications may lead to unwanted results since multiple ray-box intersection outputs may accumulate in the same location. Second, since the output format is sparse, computing the target location to accumulate the product from the multiplication requires relatively complex logic that the existing intersection test pipeline cannot handle. Therefore, `RT+SpMSpM` makes the following major design decisions to support accumulation in ray tracing hardware.

First, `RT+SpMSpM` enforces the scheduler to map rays generated from the same row to the same ray tracing unit. As all rays from the same row go to one dedicated ray tracing unit, only one ray tracing unit can contribute to the computation of each row in the result. In addition, the ray generation function extracts non-zero elements from the row-order encoded CSR format, so each ray tracing unit will likely work for only a specific row at a given time window. Since all ray tracing units can still work on different rows simultaneously and the number of rows from datasets in our target applications is significantly more than the total amount of ray tracing units in modern GPUs (i.e., 128 in NVIDIA's RTX 4090), the enforcement in scheduling does not sacrifice parallelism.

Second, `RT+SpMSpM` adds a hardware accumulation engine that asynchronously visits uncommitted outputs from the result buffer atomically. Figure 11 depicts the design of the accumulation engine. The accumulation engine has the following main components: a row hash, a buffer walking logic, and a commit/row switch logic. Since a ray tracing unit will likely work on a specific row at a given window, the accumulation engine uses a row hash to cache results from the non-zero resulting elements. The row buffer design is similar to the representation of a row in the CSR format, as each row buffer entry stores the column index and the corresponding result value. The walking logic oversees both the result buffer and the row cache. When the intersection test pipeline generates a new result element, the walking logic checks the ray's ID, its x-coordinate from the source matrix, and the y-coordinate of the hit-box to determine the hashed location of the accumulation result. In our design, we ensure the number of entries as a power of two. Therefore, we can simply use a bitmask as a hash, and linearly walk if a result element occupies the the hashed entry. If the ray's ID hits the current row ID of the row hash, the logic will accumulate the result directly in the row hash. Otherwise, the accumulation engine will commit the current row content to the target memory location reserved for the current row that the buffer stores and fetch the current row content of the ray's ID to the buffer. Suppose the hash targets a location beyond the buffer's current entry. In that case, the engine also needs to perform memory operations to write back the current row buffer and fetch the unbuffered part of the row to the row buffer.

The proposed architecture also requires minor modifications to the software to support it. First, to the ease of managing the row hash, a cache of a result row, the software must pre-allocate memory locations for each row in the result matrix at the length of the most non-zero elements in a column from the matrix used to build the acceleration structure and pass along with the ray information. Second, the software will have to re-permutate and condense the row entries at the end of the execution.

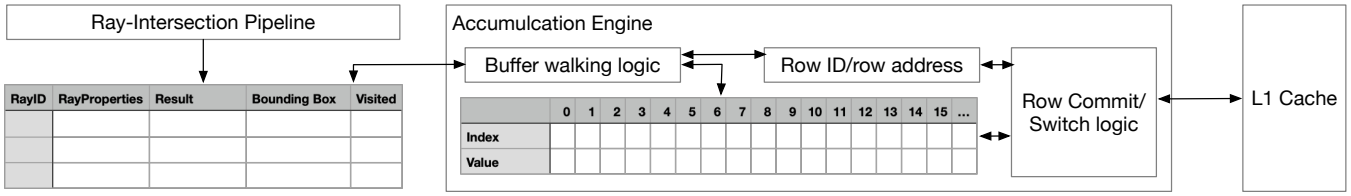


Figure 11: The proposed accumulation engine

The design brings the following advantage to RT+SpMSpM that SW-RTSpMSpM cannot easily enjoy. First, the accumulation engine with the row buffer significantly reduces the pressure on the memory. The buffer groups update to a row only when a row switch or the buffer overflows. In contrast, SWRTSpMSpM must always commit results and perform atomic add operations that access memory. The second one is more significant. By forcing the ray tracing unit to generate results for a row at a given execution window, the data access pattern automatically exploits the most efficient Gustavson's dataflow for SpMSpM [16].

6 Results of RT+SpMSpM

By proposing the architectural support that adds 0.2% area overhead to a GPU core, RT+SpMSpM achieves 1.66× to SW-RTSpMSpM, making the overall speedup to 3.06× compared with cuSPARSE

6.1 Methodology

This paper evaluated RT+SpMSpM in the following aspects. (1) We evaluated the latency and area overhead of the proposed architectural changes by realizing the RTL circuit design and synthesization using the Synopsys Design Compiler and the 45 nm technology library. (2) We produce the trace of coordinates and memory accesses through instrumenting the SW-RTSpMSpM. Then, we used a custom trace-based simulator to perform behavioral simulations that emulate the effects of resource contentions in RT cores units, the proposed buffers, and the memory accesses. (3) We extended SW-RTSpMSpM with conditional overhead that adds latency and synchronization primitives (e.g., to emulate the write backs of buffer content) to the program based on the behavioral simulation result. As our emulation methodology collects performance data on real hardware, the proposed approach can more accurately measure the performance of the proposed architecture in real applications since this method can faithfully reflect the undocumented hardware acceleration and scheduling optimizations in modern hardware.

We use a 128-RT-Cores-equipped NVIDIA's RTX 4090 GPU in the evaluations as the software implementation used in Section 4.3. Each accumulation engine in the RT Cores can store 1K entries (i.e., an 8KB data array) in the row buffer. The simulator models the mapping of rays to these 128 RT Cores and their effects on the 8KB row buffer to produce the memory traces to the L1 cache.

6.2 Area and latency overhead

Under the identical hardware configurations as our performance simulation and emulation, RT+SpMSpM architecture adds tiny area overhead, a negligible 0.21%, to a consumer-grade, high-performance GPU. We estimated RT+SpMSpM's area from the RTL code and synthesized it with the Synopsys Design Compiler using the 45 nm technology library. By scaling the design to the TSMC N5 process can compare the area overhead with NVIDIA's AD102 [30], RT+SpMSpM only adds 1.14% area overhead to each ray tracing unit. The 8K-entry buffer per accumulation engine contributes to more than 80% of the area overhead. The total overhead of RT+SpMSpM a total of 0.21% area to the 609 mm^2 chip.

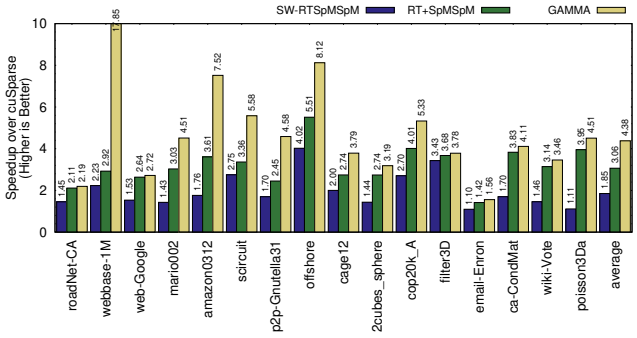


Figure 12: The experimental result of RT+SpMSpM, SW-RTSpMSpM, and GAMMA, compared to cuSPARSE

As none of the proposed architectural optimizations are located at the critical stage of the ray-intersection test pipeline, the synthesized result shows the proposed design does not increase the ray-intersection cycle time but does not result in additional stages. Therefore, the proposed architecture enables software emulation for accurate performance estimation.

6.3 Performance

Figure 12 shows the speedup of RT+SpMSpM using cuSPARSE on the same hardware as the baseline. RT+SpMSpM achieves an average 3.06× speedup compared to cuSPARSE, and 1.66× speedup over the previously presented SW-RTSpMSpM. RT+SpMSpM outperforms cuSPARSE the most at 5.51× when using offshore dataset, same as SW-RTSpMSpM. Even with the least performing case, email-Enron, RT+SpMSpM still achieves 1.42× speedup. The presented result includes the additional pre-processing and post-processing overhead when building acceleration structures and realigning result elements.

We also included a state-of-the-art SpMSpM accelerator, GAMMA [62], as another reference design. Since we used the same dataset as GAMMA [62] and both compared to intel MKL, we scaled the performance of GAMMA to the emulated hardware platform Figure 12. On average, the proposed RT+SpMSpM achieves roughly 70%

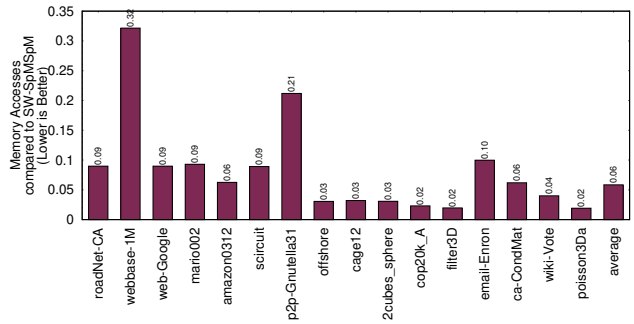


Figure 13: The reduction of memory pressure of RT+SpMSpM, compared to SW-RTSpMSpM

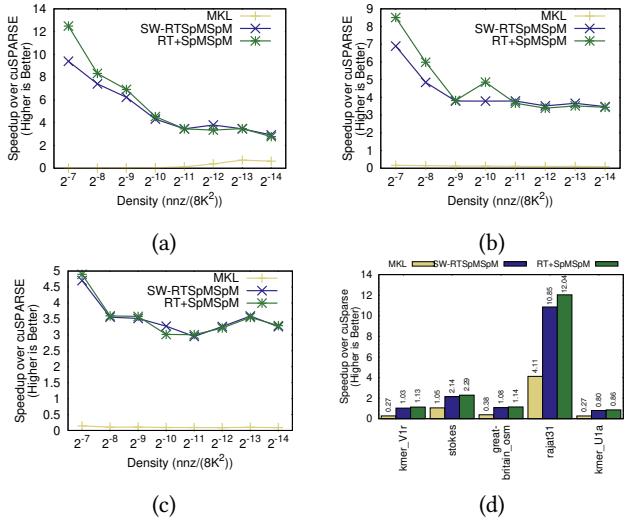


Figure 14: The performance of RT+SpMSpM and SW-RTSpMSpM, compared to intel MKL and cuSparse under different densities, shapes (a),(b),(c) and large datasets (d).

of GAMMA's performance, but RT+SpMSpM supports more than SpMSpM and shares the same instruction frontend as existing GPU architectures without relying on a separate memory subsystem design.

6.4 Improvement on Memory Accesses

The key problem that limits the performance of SW-RTSpMSpM is the amplification of memory accesses. Figure 13 compares the memory accesses through simulation. RT+SpMSpM's design in eliminating shader functions and buffering row accumulations significantly reduces the memory accesses – 94% on average. The simulation result also includes the memory accesses that the row buffer overflow contributes and provides inputs for the performance emulation that Figure 12 presents. With 1K entries, only webbase-1M and email-Enron will suffer from the case where we need to switch a single row multiple times. webbase-1M is also the case where GAMMA significantly outperforms RT+SpMSpM.

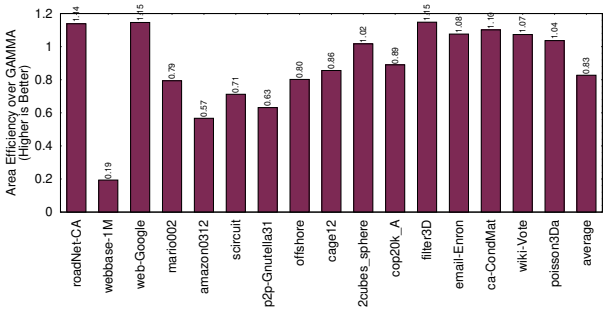


Figure 15: The area-efficiency of RT+SpMSpM, compared to GAMMA

6.5 Densities, distributions, and larger graphs

In most cases, RT+SpMSpM can outperform the state-of-the-art sparse GPU library, cuSPARSE, accross various densities, sizes, and data distributions. Figure 14(a)-(c) compares the performance of RT+SpMSpM with various densities using 8K by 8K sparse matrices and different worst-case non-zero elements within a row/column. We do not demonstrate the case where density is higher than 1% as the state-of-the-art cuSPARSE does not show advantages over tensor-core-accelerated GEMM library above that density on our hardware. We also generate matrices with the worst-case row densities, where the densities row can contain up to (a) 12.5% (b) 25% (c) 50% elements of the row size, implying the worst-case depth of the formed BVH tree in ray-tracing. Comparing the performance of different densities in Figure 14(a)-(c), when density is higher, the advantage of RT+SpMSpM is more significant over cuSPARSE as RT cores help coalesce memory accesses and mitigate control divergence. Due to the reduced amount of memory accesses and computation, the advantage of RT+SpMSpM becomes less significant but still able to outperform cuSPARSE. For different worst-case row densities, the speedup of RT+SpMSpM is less significant when the worst-case row increases.

Figure 14(d) shows the performance of RT+SpMSpM when using datasets with more than 4 million rows from [9]. RT+SpMSpM still outperforms cuSPARSE except for kmer_U1a, where the dataset has the highest worst-case row density among all datasets.

6.6 Area Efficiency

Figure 15 plots the average relative performance-per-area comparison compared with a similar-sized GAMMA, the dedicated SpMSpM accelerator. When calculating the performance-per-area, RT+SpMSpM includes the area of each entire ray tracing unit logic that performs normal ray tracing operations. Despite containing logics that SpMSpM does not use, RT+SpMSpM still achieves a very competitive 80% area-efficiency compared with GAMMA.

As RT-Cores-like design is common in modern architectures, and it shares the same memory subsystem and instruction frontend, the real area overhead is a lot smaller than any standalone SpMSpM accelerator like GAMMA. From another perspective, for the same area budget, RT+SpMSpM provides a design that supports both ray tracing and SpMSpM, but a standalone SpMSpM accelerator can only support one domain.

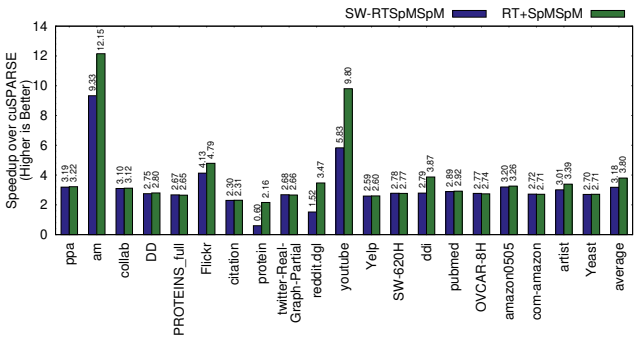


Figure 16: The speedup of RT+SpMSpM in a real graph neural networks training framework.

6.7 Power consumption and energy

To evaluate the impact of power and energy consumption of the proposed architecture, we measured power consumption when running experiments using cuSPARSE and SW-RTSpMSpM on the RTX 4090 and consistently read power sensors on the card. The average power consumption is 43.7 W for cuSPARSE and 42.5 W for SW-RTSpMSpM. SW-RTSpMSpM enjoys a 2.5% power reduction, almost the same as the baseline. As RT+SpMSpM only increases 0.2% hardware overhead, we envision the power consumption would remain the same. Therefore, the proposed software solution and the hardware architecture will reduce energy consumption in proportion to the improved execution time.

6.8 Case study: MaxK-GNN

As SpMSpM plays a significant role in real applications, we also integrated RT+SpMSpM in a graph neural networks training framework, MaxK-GNN [48]. We used the same workloads as the original MaxK-GNN paper and compared our performance. RT+SpMSpM shows an average 3.8× speedup compared to the baseline implementation that uses cuSPARSE. Even without architectural supports in RT+SpMSpM, the SW-RTSpMSpM still allows an 3.18× speedup over the baseline.

7 Related Work

Beyond the research discussed in Section 2.3, this section discusses other related work.

Utilizing Ray Tracing Hardware Beyond Graphics. Besides the SpMSpM that this paper tries to accelerate through mapping the algorithm to the ray tracing model, prior work has demonstrated the strong promise of using ray tracing as a programming model for applications from other domains, such as neighbor search [6, 34, 65]. The software optimization method that prior work discovers, such as minimizing control flow divergence, indeed benefit the implementation of SW-RTSpMSpM. However, most prior work only focuses on mapping the algorithm without considering the different nature of mapped algorithms and ray tracing, the resulting algorithm's performance is still not reaching the roofline of the existing hardware.

Despite the hardware capability of traversing the tree-structured acceleration structure, existing ray tracing hardware still cannot support more generic hierarchical tree structures like B-trees. Similar to the idea of RT+SpMSpM, prior work also exploits the route of extending the functionality of ray-intersection pipelines [1, 17]. However, unlike previous work that requires extending the ALU functions or reorganizing the pipeline, RT+SpMSpM does not use additional ALUs, does not change the cycle time of the original ray tracing operations and leads to significantly lower hardware and software overhead.

Advances in Bounding Volume Hierarchy Construction. Bounding volume hierarchies (BVHs) remain a focus of optimization [27, 33] in ray tracing, with innovations in construction algorithms, such as parallel top-down algorithms [51], bottom-up algorithms [15, 31], incremental constructuions [7, 8], and LBVH [23, 26], approaches to improve construction speed and BVH quality [13, 26, 32, 49, 58]. Recent papers have also shown works on using specific hardware for BVH constructions to improve construction speed furthermore [29, 37, 53, 56, 57], thus achieving real-time ray tracing in highly complex dynamic scene. RT+SpMSpM is orthogonal to this line of research, and RT+SpMSpM can leverage the advancement from this precious research outcome to improve the 39% of time once they are implemented in commercial products or open-sourced.

Optimization of Intersection Tests. Ray intersection tests serve as a major bottleneck of the ray tracing pipeline due to the intensity in computation [11, 12, 33]. Prior work presents architectural optimizations, including innovative intersection unit design [2, 24, 38, 55], efficient memory schedulers[25, 40, 47, 54], or intersection predictors [28]. Compared with conventional compute-intensive ray-tracing problems, RT+SpMSpM deals with the more memory-bounded SpMSpM problem. Therefore, the proposed solution of RT+SpMSpM naturally distinguishes from previous work in accelerating the computation bottleneck in ray tracing.

8 Conclusion

This paper presents a new algorithm that solves the critical sparse matrix multiplication (SpMSpM) problems using the ray tracing programming model. Doing so allows SpMSpM to exploit the ray tracing hardware, an accelerator aiming to mitigate control flow divergence and irregular memory access patterns. The pure software implementation using existing ray tracing hardware, SW-RTSpMSpM, shows the rich potential of 1.85× speedup over the famous cuSPARSE library. The software implementation also shows emerging bottlenecks in amplified and uncoordinated memory access. Therefore, this paper proposed RT+SpMSpM architectural optimizations in reclaiming the unused multipliers and adding an accumulation engine to perform SpMSpM entirely on ray tracing hardware. The hardware optimization delivers 3.06× speedup, and the performance-per-area closed to dedicated SpMSpM accelerators.

Acknowledgments

The authors would like to thank the anonymous reviewers for their helpful comments.

This work is supported by the National Science Foundation under Grant No.: 2231877 (https://www.nsf.gov/awardsearch/showAward?AWD_ID=2231877).

References

[1] Barnes Aaron, Shen Fangjia, and Rogers Tim. 2024. Extending GPU Ray-Tracing Units for Hierarchical Search Acceleration. In *The 57th IEEE/ACM International Symposium on Microarchitecture (MICRO 2024)*.

[2] Timo Aila and Tero Karras. 2010. Architecture considerations for tracing incoherent rays. In *Proceedings of the Conference on High Performance Graphics*. 113–122.

[3] AMD. 2023. RDNA3 Instruction Set Architecture: Reference Guide. https://www.amd.com/content/dam/amd/en/documents/radeon-tech-docs/instruction-set-architectures/rdna3-shader-instruction-set-architecture-feb-2023_0.pdf.

[4] Cleve Ashcraft and Roger G Grimes. 1999. SPOOLES: An Object-Oriented Sparse Matrix Library.. In *PPSC*. Citeseer.

[5] Nithya Attaluri. 2023. *Tardigrade: A Hardware Accelerator for Sparse Matrix Multiplication and Sparse Convolution*. Ph. D. Dissertation. Massachusetts Institute of Technology.

[6] Aaron Barnes, Fangjia Shen, and Timothy G Rogers. [n. d.]. Extending GPU Ray-Tracing Units for Hierarchical Search Acceleration. ([n. d.]).

[7] Jiří Bittner, Michal Hapala, and Vlastimil Havran. 2013. Fast insertion-based optimization of bounding volume hierarchies. In *Computer Graphics Forum*, Vol. 32. Wiley Online Library, 85–100.

[8] Jiři Bittner, Michal Hapala, and Vlastimil Havran. 2015. Incremental BVH construction for ray tracing. *Computers & Graphics* 47 (2015), 135–144.

[9] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (Dec. 2011), 25 pages. https://doi.org/10.1145/2049662.2049663

[10] Jack Dongarraxz, Andrew Lumsdaine, Xinhiu Niu, Roldan Pozoz, and Karin Remingtonx. 1994. A sparse matrix library in C++ for high performance architectures. In *Second object oriented numerics conference*. Citeseer, 214–218.

[11] Michael J Doyle, Colin Fowler, and Michael Manzke. 2012. Hardware accelerated construction of sah-based bounding volume hierarchies for interactive ray tracing. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. 209–209.

[12] Michael J Doyle, Colin Fowler, and Michael Manzke. 2013. A hardware unit for fast SAH-optimised BVH construction. *ACM Transactions on Graphics (TOG)* 32, 4 (2013), 1–10.

[13] Per Ganestam, Rasmus Barringer, Michael Doggett, and Tomas Akenine-Möller. 2015. Bonsai: rapid bounding volume hierarchy generation using mini trees. *Journal of Computer Graphics Techniques* 4, 3 (2015), 23–42.

[14] Ashish Gondimalla, Noah Chesnut, Mithuna Thottethodi, and T. N. Vijaykumar. 2019. SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 151–165. https://doi.org/10.1145/3352460.3358291

[15] Yan Gu, Yong He, Kayvon Fatahalian, and Guy Blelloch. 2013. Efficient BVH construction via approximate agglomerative clustering. In *Proceedings of the 5th High-Performance Graphics Conference*. 81–88.

[16] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (Sept. 1978), 250–269. https://doi.org/10.1145/355791.355796

[17] Dongho Ha, Lufei Liu, Yuan Hsi Chou, Seokjin Go, Won Woo Ro, Hung-Wei Tseng, and Tor M. Aamodt. 2024. Generalizing Ray Tracing Accelerators for Tree Traversals on GPUs. In *The 57th IEEE/ACM International Symposium on Microarchitecture (MICRO 2024)*.

[18] Mohamadhadi Habibzadeh, Omid Rajabi Shishvan, and Tolga Soyata. 2018. CUDA Libraries. In *GPU Parallel Program Development Using CUDA*. Chapman and Hall/CRC, 383–395.

[19] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO) (MICRO '52)*. 319–333.

[20] Guyue Huang, Zhengyang Wang, Po-An Tsai, Chen Zhang, Yufei Ding, and Yuan Xie. 2023. RM-STC: Row-Merge Dataflow Inspired GPU Sparse Tensor Core for Energy-Efficient Sparse Acceleration. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 338–352.

[21] Norman P Jouppi, Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. A Domain-specific Supercomputer for Training Deep Neural Networks. *Commun. ACM* 63, 7 (2020), 67–78.

[22] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto, ON, Canada) *(ISCA '17)*. 1–12.

[23] Tero Karras. 2012. Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. In *Proceedings of the Fourth ACM SIGGRAPH/Eurographics Conference on High-Performance Graphics*. 33–37.

[24] Daniel Kopta, Konstantin Shkurko, Josef Spjut, Erik Brunvand, and Al Davis. 2013. An energy and bandwidth efficient ray tracing architecture. In *Proceedings of the 5th High-Performance Graphics Conference*. 121–128.

[25] Daniel Kopta, Konstantin Shkurko, Josef Spjut, Erik Brunvand, and Al Davis. 2015. Memory considerations for low energy ray tracing. In *Computer Graphics Forum*, Vol. 34. Wiley Online Library, 47–59.

[26] Christian Lauterbach, Michael Garland, Shubhabrata Sengupta, David Luebke, and Dinesh Manocha. 2009. Fast BVH construction on GPUs. In *Computer Graphics Forum*, Vol. 28. Wiley Online Library, 375–384.

[27] Gábor Liktor and Karthikeyan Vaidyanathan. 2016. Bandwidth-efficient BVH layout for incremental hardware traversal.. In *High Performance Graphics*. 51–61.

[28] Lufei Liu, Wesley Chang, Francois Demoullin, Yuan Hsi Chou, Mohammadreza Saed, David Pankratz, Tyler Nowicki, and Tor M. Aamodt. 2021. Intersection Prediction for Accelerated GPU Ray Tracing. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*. 709–723. https://doi.org/10.1145/3466752.3480097

[29] Xingyu Liu, Yangdong Deng, Yufei Ni, and Zonghui Li. 2015. FastTree: A hardware KD-tree construction acceleration engine for real-time ray tracing. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1595–1598.

[30] Locuza. 2024. AD102 Die Photo. https://locuza.substack.com/p/nvidias-ad102-officially-revealed.

[31] Daniel Meister. 2018. *Bounding Volume Hierarchies for High-Performance Ray Tracing*. Ph. D. Dissertation. Czech Technical University.

[32] Daniel Meister and Jiří Bittner. 2016. Parallel BVH construction using k-means clustering. *The Visual Computer* 32 (2016), 977–987.

[33] Daniel Meister, Shinji Ogaki, Carsten Benthin, Michael J Doyle, Michael Guthe, and Jiří Bittner. 2021. A survey on bounding volume hierarchies for ray tracing. In *Computer Graphics Forum*, Vol. 40. Wiley Online Library, 683–712.

[34] Nate Morrical, Ingo Wald, Will Usher, and Valerio Pascucci. 2020. Accelerating unstructured mesh point location with RT cores. *IEEE transactions on visualization and computer graphics* 28, 8 (2020), 2852–2866.

[35] Francisco Muñoz-Martínez, Raveesh Garg, Michael Pellauer, José L Abellán, Manuel E Acacio, and Tushar Krishna. 2023. Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 252–265.

[36] Yuta Nagahara, Jiale Yan, Kazushi Kawamura, Masato Motomura, and Thiem Van Chu. 2024. Sparse-Sparse Matrix Multiplication Accelerator on FPGA featuring Distribute-Merge Product Dataflow. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 785–791.

[37] Jae-Ho Nah, Hyuck-Joo Kwon, Dong-Seok Kim, Cheol-Ho Jeong, Jinhong Park, Tack-Don Han, Dinesh Manocha, and Woo-Chan Park. 2014. RayCore: A ray-tracing hardware architecture for mobile devices. *ACM Transactions on Graphics (TOG)* 33, 5 (2014), 1–15.

[38] Jae-Ho Nah, Jeong-Soo Park, Chanmin Park, Jin-Woo Kim, Yun-Hye Jung, Woo-Chan Park, and Tack-Don Han. 2011. T&I engine: Traversal and intersection engine for hardware accelerated ray tracing. In *Proceedings of the 2011 SIGGRAPH Asia Conference*. 1–10.

[39] Maxim Naumov, L Chien, Philippe Vandermersch, and Ujval Kapasi. 2010. Cusparse library. In *GPU Technology Conference*, Vol. 12.

[40] Yufei Ni, Yangdong Deng, and Zonghui Li. 2022. Agglomerative Memory and Thread Scheduling for High-Performance Ray-Tracing on GPUs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 2 (2022), 334–345. https://doi.org/10.1109/TCAD.2021.3058910

[41] NVIDIA 2020. NVIDIA A100 Tensor Core GPU Architecture. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf.

[42] NVIDIA Corporation. 2023. NVIDIA Ada GPU Architecture. https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf.

[43] NVIDIA Corporation. 2024. CUSPARSE API supported by HIP. https://rocm.docs.amd.com/projects/HIPIFY/en/docs-5.2.3/tables/CUSPARSE_API_supported_by_HIP.html.

[44] NVIDIA Corporation. 2024. NVIDIA OptiX 8.1. https://raytracing-docs.nvidia.com/optix8/index.html.

[45] Toluwanimi O Odemuyiwa, Hadi Asghari-Moghaddam, Michael Pellauer, Kartik Hegde, Po-An Tsai, Neal C Crago, Aamer Jaleel, John D Owens, Edgar Solomonik,

Joel S Emer, et al. 2023. Accelerating sparse data orchestration via dynamic reflexive tiling. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 18–32.
[46] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. Outerspace: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 724–736.
[47] Hyungman Park, Donald Fussell, and Paul Navrátil. 2022. Data-Aware Predictive Scheduling for Distributed-Memory Ray Tracing. *IEEE Transactions on Visualization and Computer Graphics* 28, 1 (2022), 1172–1181. https://doi.org/10.1109/TVCG.2021.3114838
[48] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 683–698. https://doi.org/10.1145/3620665.3640426
[49] Stefan Popov, Iliyan Georgiev, Rossen Dimov, and Philipp Slusallek. 2009. Object partitioning considered harmful: space subdivision for BVHs. In *Proceedings of the Conference on High Performance Graphics 2009*. 15–22.
[50] Mohammadreza Saed, Yuan Hsi Chou, Lufei Liu, Tyler Nowicki, and Tor M. Aamodt. 2022. Vulkan-Sim: A GPU Architecture Simulator for Ray Tracing. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. https://doi.org/10.1109/MICRO56248.2022.00027
[51] Dmitry Sopin, Denis Bogolepov, and Danila Ulyanov. 2011. Real-time SAH BVH construction for ray tracing dynamic scenes. 74–77.
[52] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product. In *the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 766–780.
[53] Yin Su, Hui Guo, Run Yan, Yong Wang, Yongwen Wang, Nong Xiao, Gang Chen, Weihua Zhang, and Libo Huang. 2024. QuickTree: A Fast Hardware BVH Construction Engine. In *Proceedings of the 21st ACM International Conference on Computing Frontiers* (Ischia, Italy) *(CF '24)*. Association for Computing Machinery, New York, NY, USA, 294–297. https://doi.org/10.1145/3649153.3649292
[54] Elena Vasiou, Konstantin Shkurko, Erik Brunvand, and Cem Yuksel. 2022. Mach-RT: A Many Chip Architecture for High Performance Ray Tracing. *IEEE Transactions on Visualization and Computer Graphics* 28, 3 (2022), 1585–1596. https://doi.org/10.1109/TVCG.2020.3021048
[55] Elena Vasiou, Konstantin Shkurko, Ian Mallett, Erik Brunvand, and Cem Yuksel. 2018. A detailed study of ray tracing performance: render time and energy cost. *The Visual Computer* 34 (2018), 875–885.
[56] Timo Viitanen, Matias Koskela, Pekka Jääskeläinen, Heikki Kultala, and Jarmo Takala. 2015. MergeTree: a HLBVH constructor for mobile systems. In *SIGGRAPH Asia 2015 Technical Briefs*. 1–4.
[57] Timo Viitanen, Matias Koskela, Pekka Jääskeläinen, Heikki Kultala, and Jarmo Takala. 2017. MergeTree: A fast hardware HLBVH constructor for animated ray tracing. *ACM Transactions on Graphics (TOG)* 36, 5 (2017), 1–14.
[58] Bruce Walter, Kavita Bala, Milind Kulkarni, and Keshav Pingali. 2008. Fast agglomerative clustering for rendering. In *2008 IEEE Symposium on Interactive Ray Tracing*. IEEE, 81–86.
[59] Endong Wang, Qing Zhang, Bo Shen, Guangyong Zhang, Xiaowei Lu, Qing Wu, Yajuan Wang, Endong Wang, Qing Zhang, Bo Shen, et al. 2014. Intel math kernel library. *High-Performance Computing on the Intel® Xeon Phi™: How to Fully Exploit MIC Architectures* (2014), 167–188.
[60] Yannan Nellie Wu, Po-An Tsai, Saurav Muralidharan, Angshuman Parashar, Vivienne Sze, and Joel Emer. 2023. Highlight: Efficient and flexible dnn acceleration with hierarchical structured sparsity. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1106–1120.
[61] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. 2022. Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1377–1395. https://doi.org/10.1109/MICRO56248.2022.00096
[62] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: leveraging Gustavson's algorithm to accelerate sparse matrix multiplication *(ASPLOS '21)*. 687–701. https://doi.org/10.1145/3445814.3446702
[63] Yunan Zhang, Po-An Tsai, and Hung-Wei Tseng. 2024. Sparsepipe: Sparse Inter-operator Dataflow Architecture with Cross-Iteration Reuse. In *The 57th IEEE/ACM International Symposium on Microarchitecture (MICRO 2024)*.
[64] Zhekai Zhang, Hanrui Wang, Song Han, and William J Dally. 2020. SpArch: Efficient architecture for sparse matrix multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 261–274.
[65] Yuhao Zhu. 2022. RTNN: Accelerating Neighbor Search Using Hardware Ray Tracing. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*. 76–89. https://doi.org/10.1145/3503221.3508409

A Artifact Appendix

A.1 Abstract

This document describes the artifact of "RT+SpMSpM: Harnessing Ray Tracing for Efficient Sparse Matrix Computations" and the process of reproducing the experimental results in this paper. To run the benchmarks that this paper evaluates, the evaluator must have a computer equipped with (1) NVIDIA's GPU with compute capability 5.0 and higher, and (2) capable of running a Linux distribution supporting the software stacks for the GPU.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Presenting a new sparse matrix algorithm using ray tracing
- **Program:** cuSparse and optix are both provided in the artifact
- **Compilation:** cmake 3.22, gcc 7.5.0
- **Data set:** Real-world datasets from SuitSparse Matrix Collection. Download script provided.
- **Run-time environment:** Ubuntu 22.04, CUDA 12.3, Docker 27.3.1
- **Hardware:** A NVIDIA GPU with compute capability 5.0 or higher. (7.5 Recommended)
- **Execution:** To reduce the disturbance from another workload, we recommend running experiments with a sole user.
- **Metrics:** latency (mili-second)
- **Output:** Each benchmark program will display its execution result through the console or log files.
- **How much disk space required (approximately)?:** Installation itself should be small, but datasets could take up to 2GB per file.
- **How much time is needed to prepare workflow (approximately)?:** 10 minutes
- **How much time is needed to complete experiments (approximately)?:** 2 - 3 hours
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** We will be using an MIT license for our code.
- **Data licenses (if publicly available)?:** The datasets are publicly available through their original licensing terms.

A.3 Description

A.3.1 How to access. We archive the source code and workloads at https://zenodo.org/record/8210452. For the latest version, the user can access our GitHub page: https://github.com/escalab/RTSpMSpM

A.3.2 Hardware dependencies. To build the SWOptixSpMSpM prototype system, the user will need the hardware components and the construction guide as previously mentions. In summary, the experimental platform contains the following hardware components.

- **Processor**: Intel Core i7 14700K
- **DRAM**: 128GB DDR4
- **GPU**: 128 RT-core Maxwell NVIDIA GPU 4090

A.3.3 Software dependencies. The RTSpmSpm artifact relies on the following software components.

- CUDA 12.3
- Docker 27.3.1

Since the RTSpmSpm artifact leverages nvidia-docker to avoid manually installing many software dependencies, the following software dependencies are required if nvidia-docker is not used during compilation.

- cmake 3.10 or newer

- Cuda compilation tools 12.3
- CUDA 12.3
- cmake 3.22
- gcc 7.5.0

A.3.4 Data sets. The datasets can be downloaded from SuitSparse Matrix Collection. The user can utilize or refer to `/home/scripts/download_dataset.sh` for more details on the specific datasets.

A.4 Installation

Before installing any RTSpmSpm software/library, the user should install the software components as Section A.3.3 mentions. Then, the user can install the RTSpmSpm artifact through the following steps.

```
git clone https://github.com/escal/RTSpMSpM
sh Dockerfile/build_image.sh
sh Dockerfile/start_image.sh
sh Dockerfile/run.sh
```

And within the docker container, do the following steps.

```
cd RTSpMSpM/scripts
./install.sh
```

This step will generate the example executable that utilizes RTSpmSpm library.

A.5 Experiment workflow

To run the example executable named `optixSpMSpM`, the user can leverage the existing shell scripts under `scripts/` called `ae_run.sh` to begin the process.

```
sh /home/RTSpMSpM/scripts/ae_run.sh
```

A.6 Evaluation and expected results

A.6.1 Evaluate Results. The evaluator can redirect the outputs to a log file and carefully examine the results.

A.6.2 Expected Results. Compared to the GPU baseline, SW-RTSpMSpM can offer 1.85x speedup. Please refer to Section 4.4 for the expected results.

A.7 Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- http://cTuning.org/ae/submission-20201122.html
- http://cTuning.org/ae/reviewing-20201122.html